

AWS DevOps EC2, ASG & ELB Implementation Report

Prepared by: Chandrashekar G

Project: End-to-End Hosting of Static Application using EC2, AMI, Launch Templates, Auto Scaling Group (ASG), and Elastic Load Balancer (ELB)

1. Overview & Architecture Diagram

Concept

The architecture consists of hosting a static web application on an initial EC2 instance using Nginx as the web server. An Amazon Machine Image (AMI) is created from this instance and configured within a Launch Template. An Auto Scaling Group (ASG) dynamically manages the instances across multiple Availability Zones, registered under a Target Group managed by an Application Elastic Load Balancer (ELB).

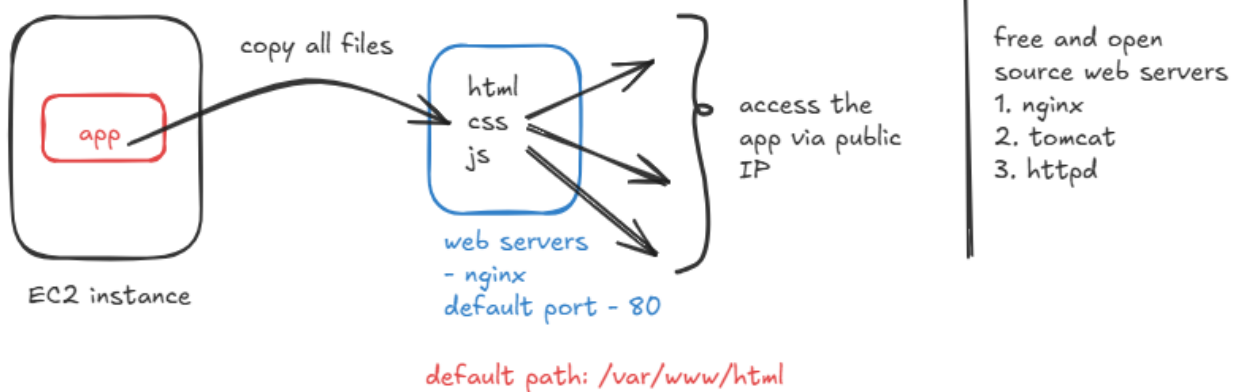
Free & Open Source Web Servers Options

- Nginx
- Apache (httpd)
- Tomcat

Default web server root path: /var/www/html

Default HTTP port: 80

How to host an application in EC2 instance ?



2. Step-by-Step Implementation Guide

Phase 1: Launch EC2 Instance & Deploy Static Application

1. Launch EC2 Instance:

- Go to the AWS Management Console -> EC2 Dashboard.
- Launch an EC2 instance with Ubuntu OS and t3.micro instance type.
- Set security group to default or allow HTTP (Port 80) and SSH (Port 22).

2. Connect & Host Website:

- Connect to the instance using EC2 Instance Connect or SSH.
- Download web template from Tooplate using wget:

```
wget -O <zip-filename>.zip <download-link-URL>
```

- Install unzip and extract files:

```
sudo apt install unzip -y  
unzip <zip-filename>.zip
```

- Install and start Nginx web server:

```
sudo apt update && sudo apt install nginx -y
```

- Verify Nginx installation:

```
nginx -v
```

- Copy extracted static website files into Nginx default root directory:

```
sudo cp -r <source-folder-path>/* /var/www/html/
```

3. Verify Deployment:

- Copy the public IP of the instance and test in browser: <http://<public-IP-instance>:80>

Phase 2: Create Custom AMI (Amazon Machine Image)

1. Navigate to Instances in the EC2 Console.
2. Select the running instance (cgin-01).
3. Click Actions -> Image and templates -> Create image.
4. Provide image name (e.g., cgimg-01) and save.

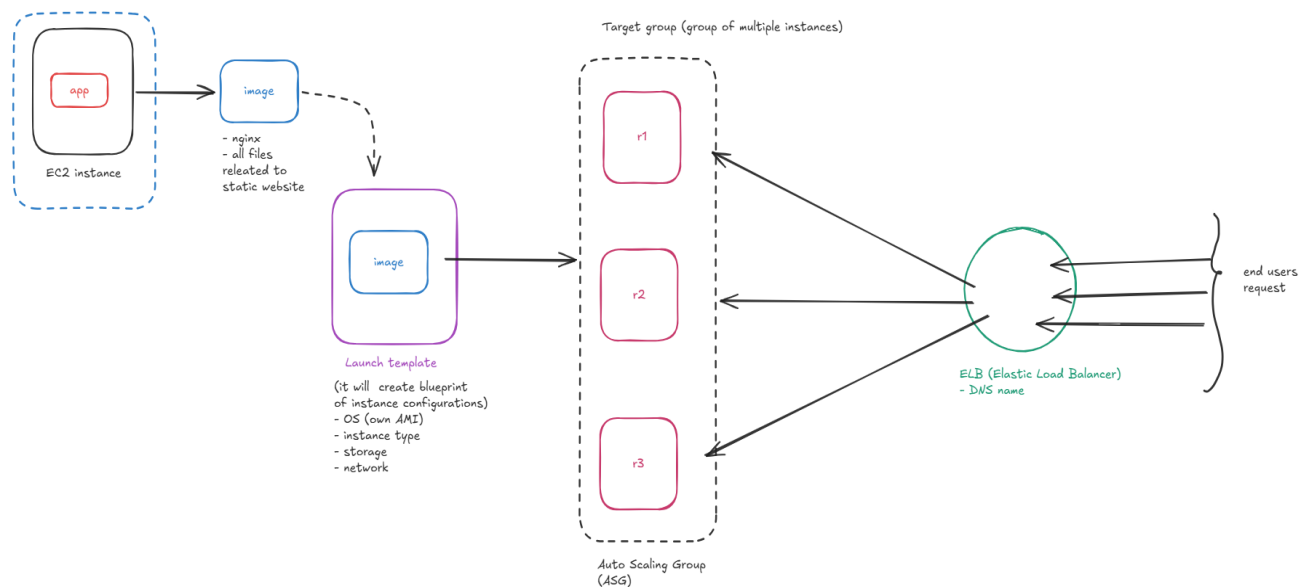
Phase 3: Create Launch Template

1. Go to EC2 Dashboard -> Launch Templates -> Create launch template.
2. Provide template name (e.g., cgtem-01).
3. Enable Auto Scaling guidance option.
4. Select the custom AMI created in Phase 2 (cgimg-01).
5. Select instance type as t3.micro.
6. Choose default security group and key pair.
7. Click Create launch template.

Phase 4: Configure Target Group, ELB, and Auto Scaling Group (ASG)

Implementation of ASG and ELB

Architecture



1. Create Auto Scaling Group:

- Go to Auto Scaling Groups -> Create Auto Scaling Group.
- Provide ASG name (e.g., cgasg-01) and choose launch template cgtem-01.
- Select 3 Availability Zones for high availability.

2. Attach Elastic Load Balancer (ELB) & Target Group:

- Choose Attach to a new load balancer.
- Select Load Balancer type: Application Load Balancer.
- Set Scheme to Internet-facing.
- Create a new Target Group (e.g., cgasg-01-1) with HTTP Port 80.
- Enable Health checks and proceed.

3. Configure Group Size & Scaling Policies:

- **Set Desired Capacity:** 1
- **Set Min Capacity:** 1
- **Set Max Capacity:** 5
- Select Target tracking scaling policy (e.g., CPU Utilization at 50%).

4. Register Targets & Resource Verification:

- Open Target Groups -> Select cgasg-01-1 -> Register targets.
- Under Load Balancers, check Resource Map to ensure active connection across Listeners, Rules, Target Groups, and Healthy Targets.

Phase 5: Access Application & Cleanup

Accessing Application:

- Copy the DNS name from the Elastic Load Balancer (e.g., cgasg-01-1-162393767.us-east-1.elb.amazonaws.com).
- Paste in browser using HTTP: `http://<load-balancer-DNS-name>`

Resource Cleanup Sequence:

To avoid unnecessary AWS charges, clean up resources in the following exact order:

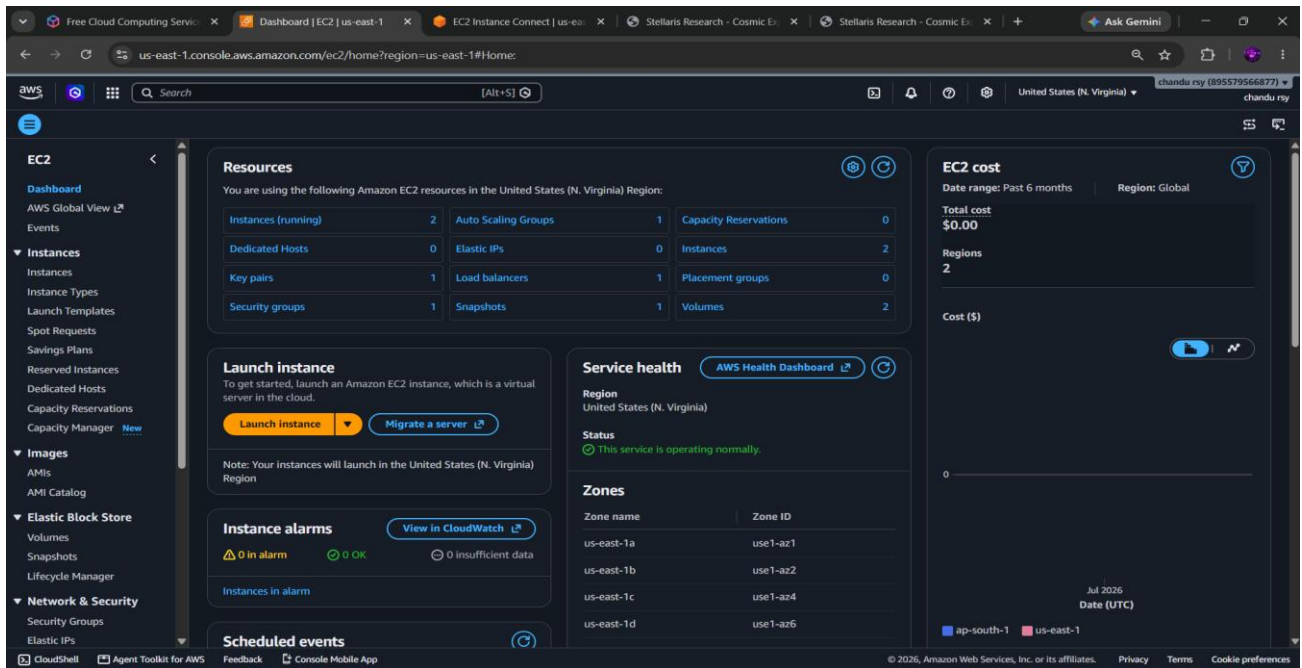
- Delete the Load Balancer
- Delete the Target Group
- Delete the Auto Scaling Group (ASG)
- Delete the Launch Template
- Deregister AMI & Delete Snapshots
- Terminate any remaining EC2 Instances

3. Project Visual Evidence & Snapshots

Below are the 11 key project verification snapshots demonstrating each stage of deployment:

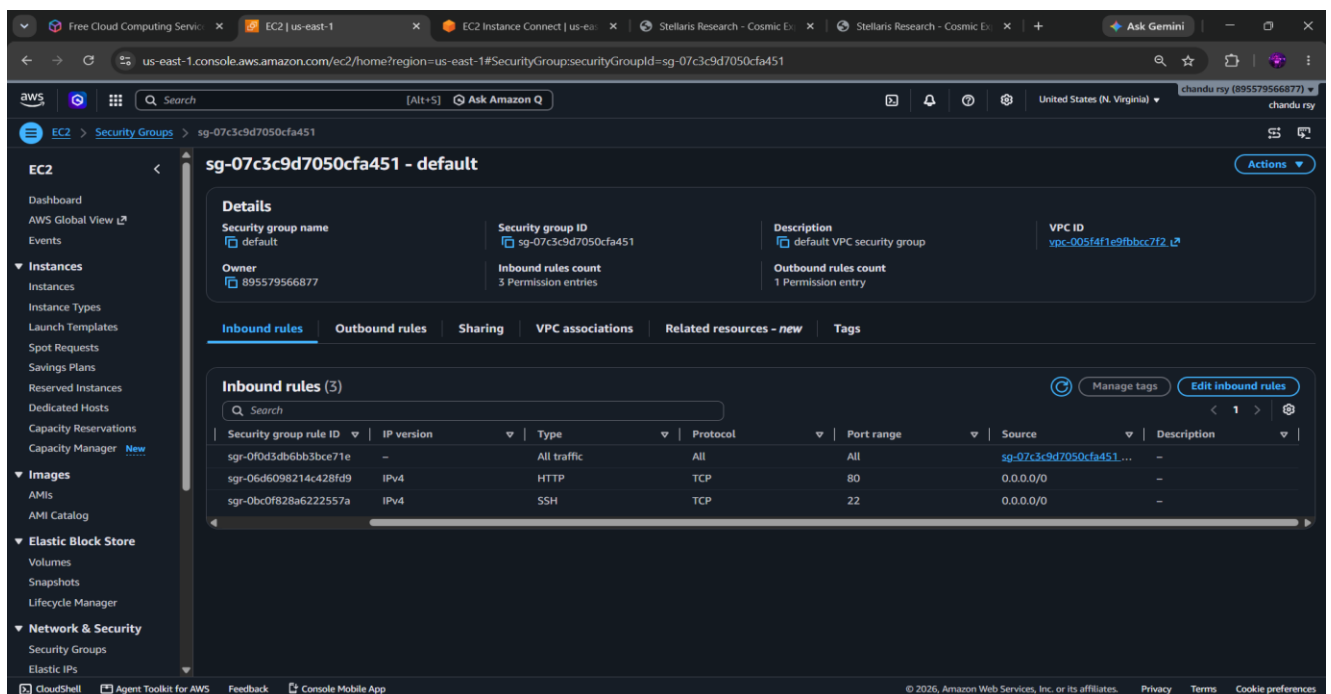
Snapshot 1: EC2 Dashboard Overview

Description: Shows active resources including 2 running instances, 1 Auto Scaling group, 1 Target Group, 1 Load Balancer, and Security Groups.



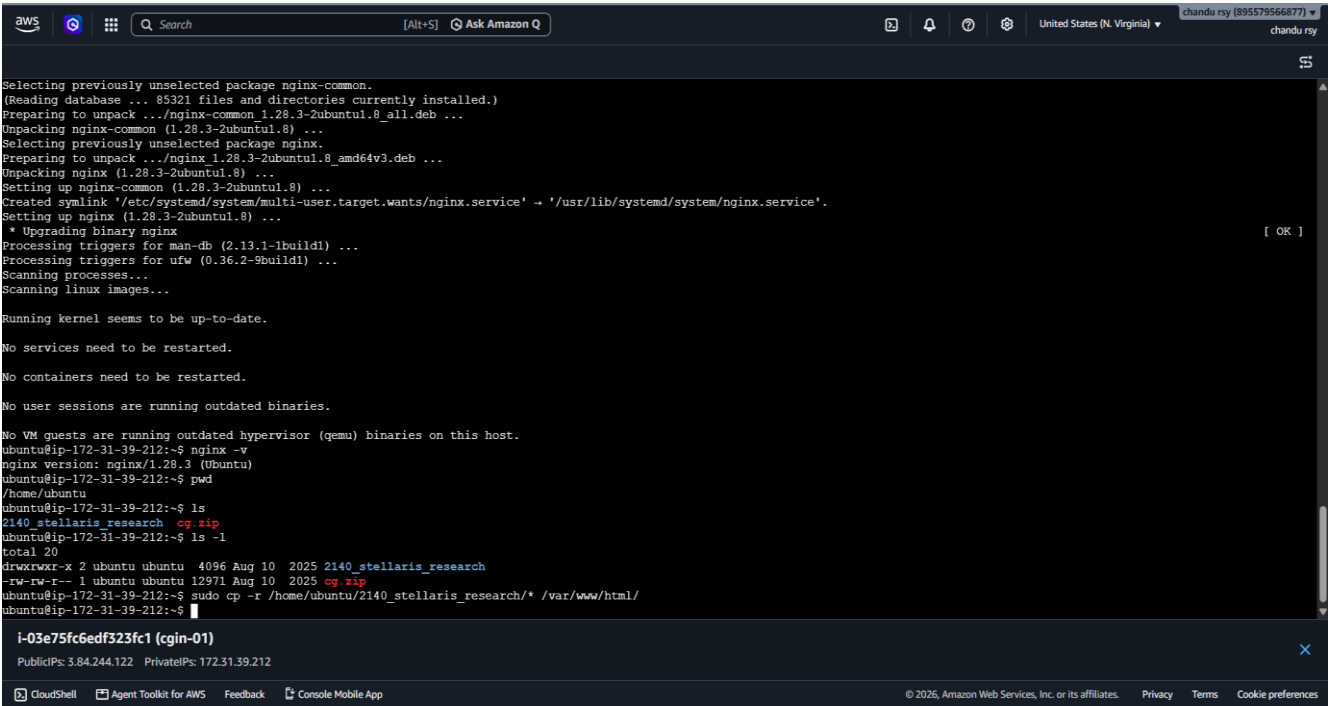
Snapshot 2: Security Group Configuration

Description: Inbound rules configured for Security Group sg-07c3c9d7050cfa451 allowing HTTP traffic on Port 80 and SSH on Port 22.



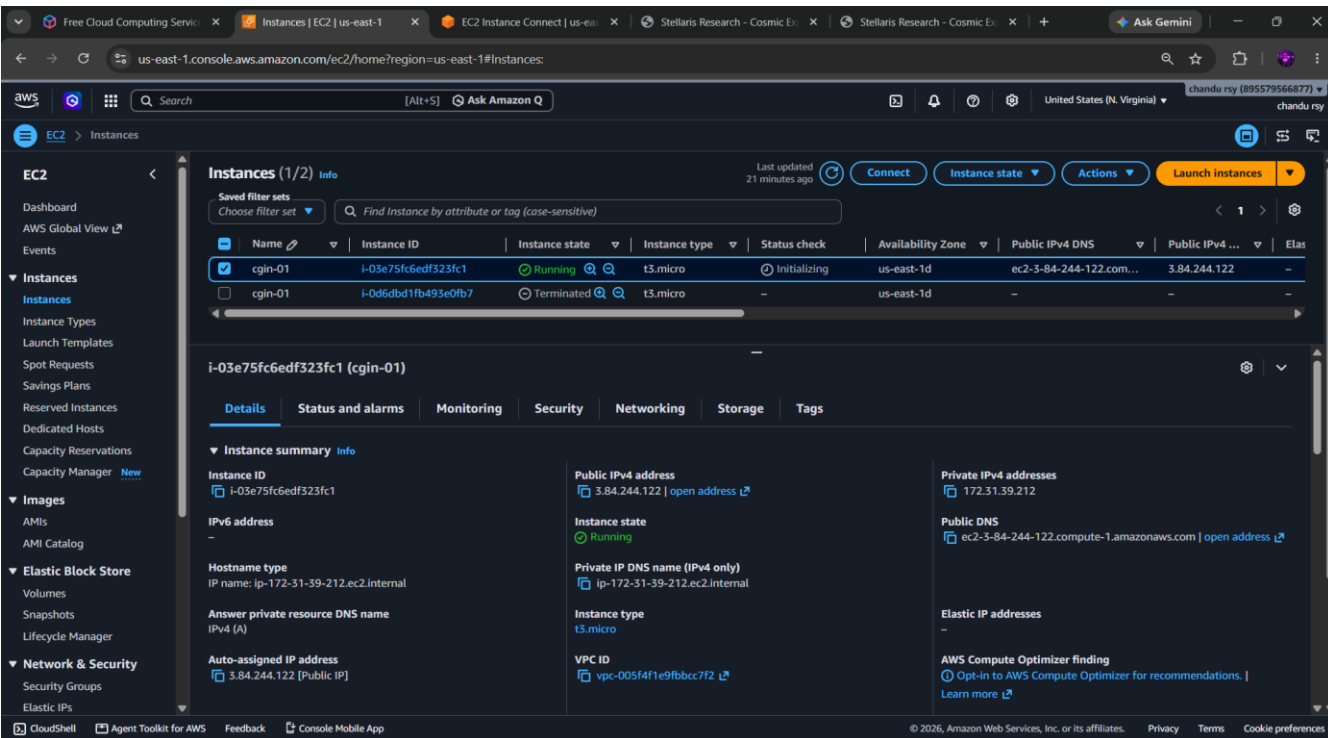
Snapshot 3: Instance Provisioning & Terminal Setup

Description: Terminal console execution showing Nginx setup, package installation, and copying static website files (2140_stellaris_research) to /var/www/html/.



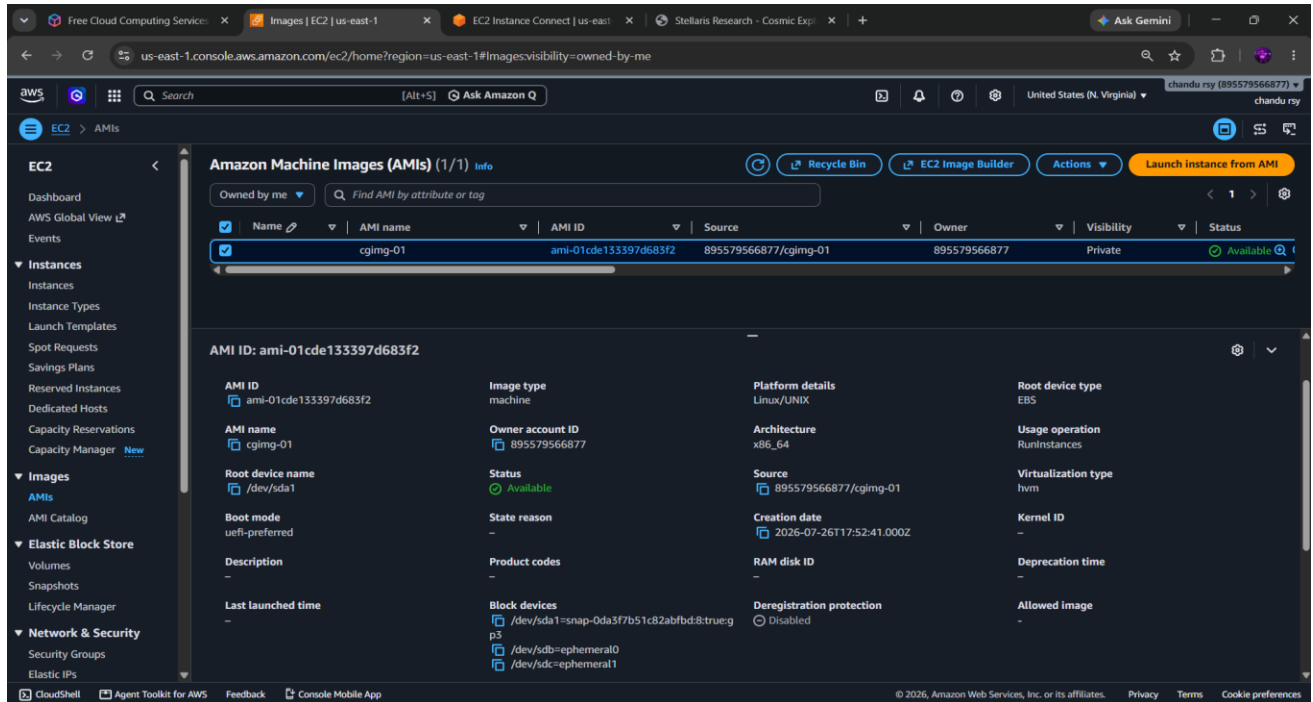
Snapshot 4: Running EC2 Instance Details

Description: View of instance cgin-01 (i-03e75fc6edf323fc1) in Running state with Public IPv4 address 3.84.244.122.



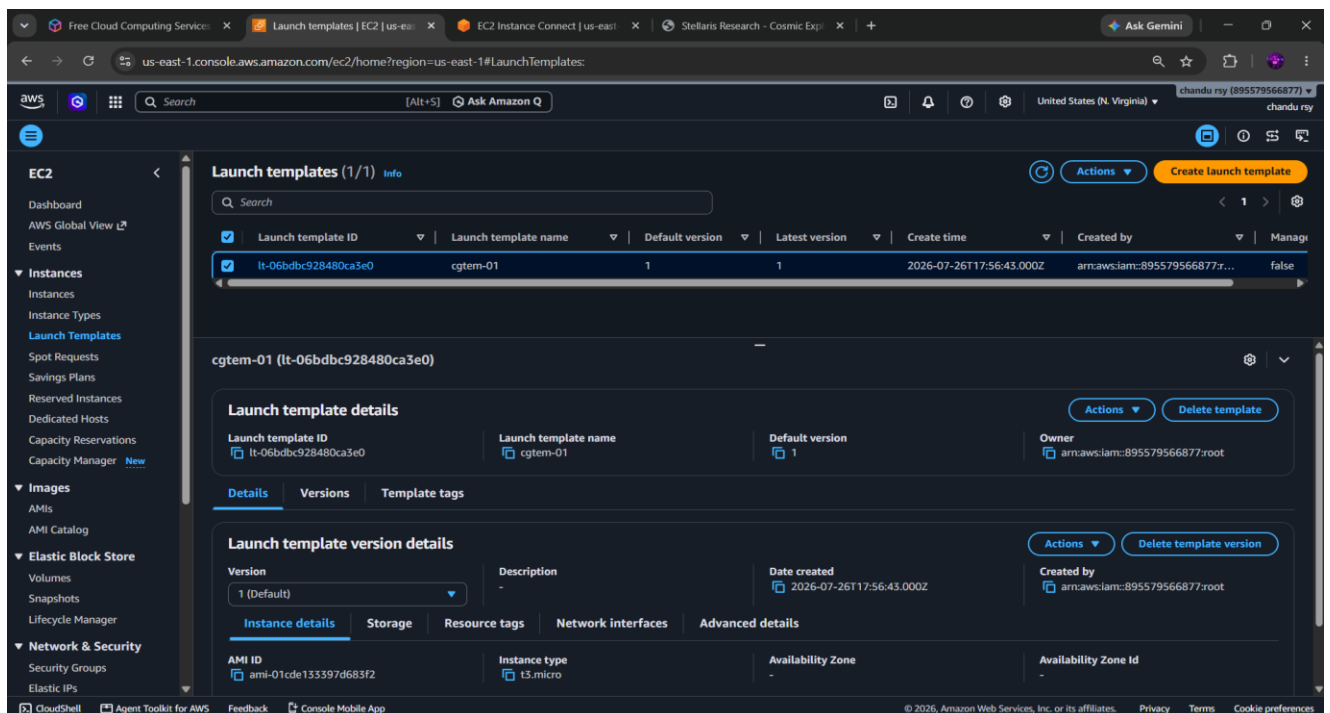
Snapshot 5: Amazon Machine Image (AMI) Creation

Description: Custom AMI created named cgimg-01 (ami-01cde133397d683f2) in Available status.



Snapshot 6: Launch Template Creation

Description: Launch template cgtem-01 (lt-06bdbc928480ca3e0) configured with custom AMI ami-01cde133397d683f2 and t3.micro instance type.



Snapshot 7: Auto Scaling Group (ASG) Setup

Description: ASG cgasg-01 created with Desired capacity: 1, Scaling limits: 1–5, attached to launch template cgttem-01.

The screenshot displays the AWS Management Console for the Auto Scaling group 'cgasg-01'. The left sidebar shows the navigation menu with categories like Capacity Reservations, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area shows the 'Auto Scaling groups (1/1)' overview with a table listing the group 'cgasg-01' with a status of 'Updating capacity'. Below this, the 'Auto Scaling group: cgasg-01' details are shown, including the 'cgasg-01 Capacity overview' and the 'Launch template' section.

Name	Status	Launch template/configuration	Instances	Instance health	Desired capacity	Min
cgasg-01	Updating capacity	cgttem-01 Version Default	0	-	1	1

Auto Scaling group: cgasg-01

cgasg-01 Capacity overview

arn:aws:autoscaling:us-east-1:895579566877:autoScalingGroup:d2160859-6b43-44cb-b627-bf39a46b7c40:autoScalingGroupName/cgasg-01

Desired capacity: 1 | Scaling limits: 1 - 5 | Desired capacity type: Units (number of instances) | Status: Updating capacity

Date created: Sun Jul 26 2026 23:30:17 GMT+0530 (India Standard Time)

Launch template

Launch template: lt-06bdbc928480ca3e0 | AMI ID: ami-01cde133397d683f2 | Instance type: t3.micro | Owner: arnaws:iam:895579566877:root

Snapshot 8: Target Group Status

Description: Target group cgasg-01-1 associated with HTTP port 80 showing registered EC2 targets.

The screenshot displays the AWS Management Console for the Target group 'cgasg-01-1'. The left sidebar shows the navigation menu with categories like Capacity Reservations, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area shows the 'Target groups (1/1)' overview with a table listing the group 'cgasg-01-1' with a status of 'Healthy'. Below this, the 'Target group: cgasg-01-1' details are shown, including the 'Details' section and the 'Distribution of targets by Availability Zone (AZ)' section.

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
cgasg-01-1	arn:aws:elasticloadbalancing:us-east-1:895579566877:targetgroup/cgasg-01-1/1e774e6178e41774	80	HTTP	Instance	cgasg-01-1	vpc-005f4f1e9fbcc7f2

Target group: cgasg-01-1

Details

arn:aws:elasticloadbalancing:us-east-1:895579566877:targetgroup/cgasg-01-1/1e774e6178e41774

Target type: Instance | Protocol : Port: HTTP: 80 | Protocol version: HTTP1 | VPC: vpc-005f4f1e9fbcc7f2

IP address type: IPv4 | Load balancer: cgasg-01-1

2 Total targets | 0 Healthy | 0 Unhealthy | 0 Unused | 2 Initial | 0 Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Snapshot 9: Application Load Balancer (ALB) Setup

Description: Internet-facing Application Load Balancer cgasg-01-1 active across 3 Availability Zones (us-east-1c, us-east-1f, us-east-1d).

The screenshot displays the AWS Management Console for the 'us-east-1' region. The 'Load balancers' page shows a table with one entry: 'cgasg-01-1', which is in an 'Active' state, of type 'application', with an 'Internet-facing' scheme, 'IPv4' IP address type, VPC ID 'vpc-005f4f1e9fbcc7f2', and is distributed across '3 Availability Zones' with security group 'sg-07c3c9d7050c'. Below the table, the 'Load balancer: cgasg-01-1' details are shown. The 'Details' section includes: Load balancer type (Application), Scheme (Internet-facing), Status (Active), Hosted zone (Z355XDQTRQ7X7K), VPC (vpc-005f4f1e9fbcc7f2), Availability Zones (subnet-0b3f457501ee7e22b in us-east-1c, subnet-03400dc55e9b32e49 in us-east-1f, and subnet-06ba11524b66e2063 in us-east-1d), Load balancer IP address type (IPv4), and Date created (July 26, 2026, 23:30 UTC+05:30). The Load balancer ARN and DNS name info are also visible.

Snapshot 10: ELB Resource Map & Target Health

Description: End-to-end routing topology map from HTTP:80 Listener -> Forwarding Rule -> Target Group cgasg-01-1 -> 2 Healthy targets (i-03e75fc6edf323fc1 and i-0a94c48da05bc28ea).

The screenshot shows the 'Unhealthy target map' for the Application Load Balancer 'cgasg-01-1'. The map illustrates the flow from the 'Listeners (1)' (HTTP:80) through 'Rules (1)' (Priority default, Forward to target group) to 'Target groups (1) info' (Instance, HTTP, cgasg-01-1). This target group contains 'Targets (2)': 'i-03e75fc6edf323fc1' (Port 80) and 'i-0a94c48da05bc28ea' (Port 80). Both targets are marked as 'Healthy'. The map also shows the 'Conditions (if)' as 'If no other rule applies'. The 'Last fetched' time is 'seconds ago' and there is an 'Export' button.

Snapshot 11: Application Live Verification

Description: Static website successfully hosted and accessed live via the Elastic Load Balancer DNS Endpoint (cgasg-01-1-162393767.us-east-1.elb.amazonaws.com).

